16-782 Hw2

Andrew Bank

October 2026

1 Results table

Note that I have augmented the given test environment with Maps 3, 4, 5, and Challenge. Challenge was a map with cumbersome obstacles, whereas the other maps were mostly open. I have also included extra statistics such as runtime statistics and cost + cost variance.

Table 1: Comprehensive performance comparison of all five motion planning implementations across all six benchmark environments (20 random collision-free queries per map, 5.0 s cutoff).

Environment	Algorithm	DOFs	Success Rate	Mean Time (s)	Median Time (s)	Max Time (s)	Path Cost (rad)
Map 1	Vanilla RRT	5	100.0%	0.017	0.016	0.031	9.91 ± 4.99
	RRT-Connect	5	100.0%	0.016	0.016	0.019	7.68 ± 2.57
	Vanilla RRT*	5	100.0%	0.172	0.170	0.191	8.84 ± 4.01
	PRM	5	100.0%	0.020	0.016	0.031	7.55 ± 3.15
	RRT* (CO)	5	100.0%	0.362	0.231	1.066	**7.01 ± 2.09**
Map 2	Vanilla RRT	3	100.0%	0.019	0.018	0.027	9.31 ± 4.93
	RRT-Connect	3	100.0%	0.017	0.017	0.021	8.42 ± 3.86
	Vanilla RRT*	3	100.0%	0.055	0.055	0.059	6.85 ± 2.48
	PRM	3	100.0%	0.035	0.036	0.044	5.36 ± 1.67
	RRT* (CO)	3	100.0%	0.252	0.211	0.643	**5.20 ± 1.98**
Map 3	Vanilla RRT	4	100.0%	0.017	0.016	0.027	6.94 ± 3.47
	RRT-Connect	4	100.0%	0.016	0.016	0.018	6.55 ± 1.96
	Vanilla RRT*	4	100.0%	0.076	0.076	0.088	7.54 ± 2.69
	PRM	4	100.0%	0.025	0.016	0.039	5.70 ± 1.72
	RRT* (CO)	4	100.0%	0.221	0.216	0.592	**5.44 ± 1.63**
Map 4	Vanilla RRT	5	85.0%	0.843	0.044	4.520	20.25 ± 11.98
	RRT-Connect	5	100.0%	0.022	0.018	0.054	17.85 ± 10.01
	Vanilla RRT*	5	75.0%	0.421	0.130	1.224	11.71 ± 5.87
	PRM	5	100.0%	0.089	0.048	0.608	18.15 ± 7.94
	RRT* (CO)	5	75.0%	1.011	0.470	3.052	**10.18 ± 5.69**
Map 5	Vanilla RRT	4	100.0%	0.017	0.017	0.020	7.03 ± 3.71
	RRT-Connect	4	100.0%	0.016	0.016	0.017	6.00 ± 1.98
	Vanilla RRT*	4	100.0%	0.171	0.169	0.193	7.01 ± 2.83
	PRM	4	100.0%	0.026	0.016	0.112	5.98 ± 2.73
	RRT* (CO)	4	100.0%	0.306	0.288	0.584	**5.17 ± 1.99**
Map Challenge	Vanilla RRT	5	80.0%	1.074	0.059	3.683	16.77 ± 5.60
	RRT-Connect	5	100.0%	0.036	0.019	0.228	20.47 ± 11.22
	Vanilla RRT*	5	70.0%	0.146	0.060	0.390	13.41 ± 5.82
	PRM	5	100.0%	0.101	0.072	0.207	17.75 ± 6.91
	RRT* (CO)	5	70.0%	0.375	0.256	0.892	**12.33 ± 5.64**

2 Analysis

What planner do you think is the most suitable for the environment and why?

For the "simple" environments, which are Maps 1, 2, 3, and 5, every algorithm has a 100% success rate and comfortably meets the 5-second threshold runtime. Therefore, the most appropriate algorithm is the one that yields the least cost, which is the RRT* combined algorithm on every map.

However, on the more complex maps, runtimes take considerably longer and some of the algorithms start experiencing failures. Both RRT-Connect and PRM are more resistant to the failures, have faster runtimes, and still yield acceptable path costs. In cases where it doesn't fail though, RRT* combined still yields better path costs.

What issues does the planner still have?
How do you think you can improve that planner?

Vanilla RRT: Appears to be a strictly worse version than RRT-Connect. RRT's niche appears to be runtime at the expense of path cost, but RRT-connect has near-identical runtime, near-identical cost, and better performance on 'more difficult' environments.
Vanilla RRT could likely find a niche in environments that are extremely open by leveraging high goal biasing.

RRT-Connect: Experiences very high variability in path cost on 'more difficult' environments.
RRT-Connect could benefit heavily from path post-processing.

RRT*: Struggled with runtime. This clearly comes from the rewiring that it needs to do. Additionally, RRT* had a chance of failure on the more difficult maps due to exhausting its iteration budget.
The most naive solution would be to just accept higher runtimes as part of the cost of doing business with RRT*. Another possibility would be to make some sort of RRT* connect (would likely lost the optimality guarantee) to improve runtime while not sacrificing much from cost.

PRM: Honestly, PRM did really well in our environments, but that was likely because all of them were relatively small and open. In a larger environment, making the roadmap would take more runtime regardless of how close the goal would be.
Making the roadmap is unavoidable. A large issue is the bottlenecks of the map since nodes need to be placed to traverse bottlenecks on the maps to guarantee completeness. Making an algorithm that biases towards clustering points around bottlenecks rather than open areas would help.